Bonici — Déploiement

Site bonici.africa · Guide de déploiement · 14 octobre 2026

Ce document s'adresse à la personne qui installe et met en service le nouveau site de Bonici Africa. Il suppose un accès administrateur à un serveur Linux et à la zone DNS du domaine, et **aucune connaissance du code**. Chaque étape dit ce qu'il faut faire et, quand cela change la décision, pourquoi.

Le site remplace l'ancienne application Angular / Django. Il comprend le site public (menus des sept restaurants, commande sans compte, suivi de commande, avis Google, bon plan, candidatures) et l'administration (**/admin**) pour la direction et les gérants de restaurant. Deux fichiers du dépôt l'accompagnent : `README.md` (développement) et `.env.example` (toutes les variables, commentées).

1. Dépôt Git

Composant	Dépôt	Branche
Site public, administration, API et base (un seul projet)	**https://github.com/D4Ron/Bonici**	main

Tout est dans un seul dépôt et se déploie d'un bloc : il n'y a ni frontend ni backend séparés à installer. Le guide lui-même et le script qui le produit sont dans `docs/`.

2. Technologies et versions

Technologie	Version	Rôle
Next.js (App Router)	16.3.8	Site public, administration et actions serveur
React	19.3	Interface
TypeScript	5.9	Langage de tout le projet
Node.js	24 (image node:24-alpine)	Exécution de l'application
PostgreSQL	16 (image postgres:16-alpine)	Base de données
Prisma	6.19.3	Accès à la base et migrations
Caddy	2 (image caddy:2-alpine)	Reverse proxy, HTTPS automatique (Let's Encrypt)
Docker / Docker Compose	Docker 24+ avec le plugin compose v2	Installation et exploitation
Zod · bcryptjs · Nodemailer	4.6 · 3.0 · 7.0	Validation · mots de passe · envoi d'emails
Sass	1.105	Feuilles de style (reprises de l'ancien site)
Vitest	4.1	Tests (prix, statuts de commande, droits)
Odoo (facultatif)	14 à 19, testé sur 18	Réception des commandes en bons de commande

3. Ce qui tourne, et ce qui est exposé

Le site tourne entièrement dans Docker, sur un seul serveur. Quatre services, dont un seul est joignable depuis Internet :

Service	Rôle	Exposé sur Internet
proxy (Caddy)	Reçoit tout le trafic, obtient et renouvelle le certificat HTTPS, transmet à l'application	**Oui — ports 80 et 443**
app (Next.js)	Le site et l'administration	Non — 127.0.0.1:3000 uniquement
db (PostgreSQL)	La base de données	Non — réseau Docker interne
migrate	À chaque démarrage : applique les migrations et charge les données de base, puis s'arrête	Non

> **Pourquoi l'application n'écoute que sur 127.0.0.1**
> Docker ouvre les ports qu'il publie **en contournant le pare-feu ufw**. Un port publié sur toutes les interfaces serait donc joignable depuis Internet même avec ufw activé — en HTTP, sans chiffrement. C'est pourquoi `compose.yaml` publie l'application sur 127.0.0.1 seulement : tout le trafic public passe par Caddy, en HTTPS.

Deux volumes Docker contiennent les données à conserver : **db_data** (la base : commandes, carte, comptes, avis) et **uploads** (photos ajoutées dans l'administration, documents d'appels d'offre, CV des candidatures). Le reste se reconstruit depuis le dépôt.

4. Prérequis serveur

Prérequis	Valeur
Système	Linux, Ubuntu 22.04 ou 24.04 LTS (VPS)
RAM	2 Go minimum, 4 Go recommandés (la construction de l'image est la phase la plus gourmande)
CPU	2 vCPU
Stockage	20 Go (images Docker, base, photos, sauvegardes locales)
Accès	SSH avec un utilisateur sudo
Ports à ouvrir	22 (SSH), 80 et 443 (web) — rien d'autre
Domaine	Accès à la zone DNS de bonici.africa
Facultatif	Un compte SMTP (emails « mot de passe oublié ») ; un accès Odoo (section 9)

5. Procédure de déploiement

5.1 — Préparer le serveur

```
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER      # puis se déconnecter / reconnecter

sudo ufw allow OpenSSH
sudo ufw allow 80
sudo ufw allow 443
sudo ufw enable
```

5.2 — Faire pointer le domaine

Chez le registrar, faire pointer le domaine vers l'adresse IP publique du serveur. À faire **avant** le premier démarrage : Caddy a besoin que le domaine mène au serveur pour obtenir le certificat HTTPS.

Type	Nom	Valeur
A	bonici.africa (ou @)	IP du serveur
A	www	IP du serveur

Vérifier depuis le serveur : `dig +short bonici.africa` doit renvoyer son IP. La propagation prend de quelques minutes à quelques heures.

> **Basculer depuis l'ancien site**
> L'ancien site est servi sur ce même domaine : la bascule a lieu au moment où le DNS change. Installer et vérifier d'abord le nouveau serveur, au besoin sur un sous-domaine de test (DOMAIN=nouveau.bonici.africa), puis changer le DNS. **Aucune donnée n'est à migrer** : l'ancienne base ne contenait aucun plat, aucune commande réelle ni aucun avis (vérifié en octobre 2026). La carte et les avis sont chargés par le nouveau site.

5.3 — Récupérer le site et le configurer

```
git clone https://github.com/D4Ron/Bonici.git bonici
cd bonici
cp .env.example .env
chmod 600 .env
nano .env
```

Le fichier `.env` porte les secrets de cette installation : il n'est jamais versionné (exclu par `.gitignore`). Les valeurs ci-dessous sont obligatoires ; toutes les autres sont décrites en annexe.

Variable	Ce qu'elle protège ou règle	Comment la produire
POSTGRES_PASSWORD	L'accès à la base de données.	`openssl rand -base64 24`
ADMIN_EMAIL **ADMIN_PASSWORD**	Le premier compte administrateur, créé au premier démarrage si aucun administrateur n'existe.	Un mot de passe provisoire d'au moins 8 caractères, transmis à son titulaire hors du fichier. À changer dès la première connexion.
SITE_URL	Les liens envoyés par email (mot de passe oublié).	`https://bonici.africa`
DOMAIN	Le domaine pour lequel Caddy demande le certificat.	`bonici.africa`

`DATABASE_URL` est ignorée en production : Docker la construit à partir de POSTGRES_*. La laisser telle quelle.

5.4 — Premier démarrage

```
docker compose -f compose.yaml --profile prod up -d --build
```

> **Toujours « -f compose.yaml » en production**
> Sans cette option, Docker ajoute automatiquement `compose.override.yaml`, la configuration de développement : rechargement à chaud, code monté depuis le disque, base de données exposée sur le port 5433. Le profil **prod** démarre Caddy, qui n'existe pas en développement.

La première construction prend 5 à 15 minutes. Suivre son avancement :

```
docker compose -f compose.yaml logs -f migrate
#   doit finir par : Seed done: 7 branches, 53 dishes in 20 categories, 59 new reviews.
docker compose -f compose.yaml logs -f proxy
#   doit contenir : certificate obtained successfully
docker compose -f compose.yaml ps
#   app : Up (healthy)
```

6. Base de données et comptes

Paramètre	Valeur
Type	PostgreSQL 16, dans le conteneur db
Nom de la base / utilisateur	POSTGRES_DB / POSTGRES_USER du .env (bonici / bonici par défaut)
Port	5432, réseau Docker interne uniquement
Création	Automatique au premier démarrage du conteneur — aucune commande à lancer
Schéma	Géré par les migrations Prisma (`prisma/migrations`), appliquées automatiquement à chaque démarrage

Données chargées au premier démarrage

- **Les 7 restaurants** avec adresse, téléphone et coordonnées GPS : Ramatou, 2 Lions, Adjololo, Tam Tam (Lomé), Aného, Kpalimé, Kara.
- **La carte** : 53 articles en 20 catégories, transcrits des menus PDF de Bonici, avec tailles et parfums (pizzas Moyenne / Grande, poisson Petit / Moyen / Grand, jus, bières…). Le Red Bull n'est proposé qu'à Aného, Kara et Ramatou, comme sur leur menu.
- **Les 59 avis Google** des fiches Ramatou, Aného et 2 Lions. Le site n'affiche que ceux de 4 étoiles et plus avec un commentaire ; la note moyenne les compte tous.
- **Le compte administrateur** défini par ADMIN_EMAIL / ADMIN_PASSWORD.

> **Le chargement ne réécrase jamais l'administration**
> Il est rejoué à chaque démarrage mais n'ajoute que ce qui manque : la carte n'est chargée que dans une base sans aucun plat, et un restaurant déjà présent n'est pas modifié. Les prix, photos et disponibilités saisis dans l'administration survivent donc aux mises à jour.

Comptes

Compte	Création	Ce qu'il voit
Administrateur	Au premier démarrage (ADMIN_EMAIL), puis Admin › Utilisateurs	Tout : commandes de tous les restaurants, carte, promotions, avis, contenus, candidatures, Odoo
Gérant de restaurant	Admin › Utilisateurs › rôle « Gérant », rattaché à un restaurant	Les commandes et la disponibilité de la carte de son seul restaurant
Client	Facultatif, par le client lui-même (Inscription)	Ses propres commandes

Il n'existe **aucun mot de passe par défaut partagé** : le seul compte créé automatiquement est celui du .env. Les clients n'ont pas besoin de compte pour commander — c'était le principal défaut de l'ancien site.

7. Contrôle avant l'ouverture au public

Un site qui démarre n'est pas un site prêt. Dans l'ordre, une fois le premier démarrage terminé :

#	Vérification	Attendu
1	Ouvrir `https://bonici.africa`	Cadenas HTTPS ; www.bonici.africa redirige vers le domaine principal
2	Ouvrir `https://bonici.africa/api/health`	`{"ok":true}`
3	Menus : choisir un restaurant, ajouter une pizza en taille Grande, commander sans compte	La commande s'affiche avec son lien de suivi
4	Se connecter à /admin avec le compte du .env	La commande test apparaît ; la passer en « Annulée »
5	Admin › Utilisateurs › votre compte : nouveau mot de passe	Le mot de passe provisoire n'est plus valable
6	Créer un compte Gérant par restaurant	Chaque gérant ne voit que son restaurant
7	« Mot de passe oublié » avec une adresse réelle	Email reçu (si SMTP configuré)
8	Mettre en place les sauvegardes (section 10) et en restaurer une	La restauration fonctionne

Informations encore attendues de Bonici

Élément	Où le saisir	État
Adresse exacte de Bonici Tam Tam (latitude / longitude)	Admin › Restaurants › Tam Tam	**À fournir**
Horaires de chaque restaurant (actuellement « Jusqu'à 23h »)	Admin › Restaurants	**À fournir**
Confirmation des tailles (pizzas, poisson)	Admin › Plats	**À confirmer**
Description de la Salade fermière	Admin › Plats	**À fournir**
Texte des CGU ; relecture de la politique de confidentialité	Pages /cgu et /confidentialite (code)	**À fournir**
Photos réelles des plats (les actuelles sont génériques)	Admin › Plats › Image	**Progressif**
Les 7 restaurants, la carte, les avis Google, les numéros	—	**Chargés**

8. Mises à jour et retour arrière

```
cd ~/bonici
# sauvegarder d'abord (section 10)
git pull
docker compose -f compose.yaml --profile prod up -d --build
```

Les migrations de la base s'appliquent toutes seules au redémarrage : il n'y a pas de commande à ne pas oublier. Pendant la reconstruction, l'ancienne version reste en ligne ; la coupure se limite au redémarrage du conteneur, quelques secondes.

Revenir à la version précédente

```
git log --oneline -5          # repérer le commit précédent
git checkout <commit>
docker compose -f compose.yaml --profile prod up -d --build
```

> **Une migration appliquée ne se défait pas**
> Revenir au code précédent ne retire pas une modification de la base déjà faite. Si une mise à jour a changé la base et pose problème, la seule voie sûre est de restaurer la sauvegarde faite juste avant — d'où la sauvegarde avant chaque mise à jour.

9. Odoo (facultatif)

Si Bonici utilise Odoo, chaque commande du site peut y être créée automatiquement comme **bon de commande** (application Ventes) : client retrouvé par son téléphone ou créé, lignes avec remises, frais de livraison, référence WEB-*n*°. Une annulation sur le site annule aussi le bon dans Odoo.

- Dans Odoo : Préférences › Sécurité du compte › **Nouvelle clé API**, pour un utilisateur ayant les droits Ventes.
- Créer deux produits : un produit générique « Plat (site web) » et « Frais de livraison ». Noter leurs ID.
- Dans `.env` : ODOO_URL, ODOO_DB, ODOO_USERNAME, ODOO_API_KEY, ODOO_FALLBACK_PRODUCT_ID, ODOO_DELIVERY_PRODUCT_ID.
- Redémarrer (`docker compose -f compose.yaml --profile prod up -d`) puis vérifier Admin › **Odoo** : « Connecté ».
- Pour un suivi fin des ventes et du stock, renseigner sur chaque plat son ID produit Odoo (Admin › Plats).

> **Les prix du site sont TTC**
> Par défaut, aucune taxe n'est ajoutée par Odoo : sinon la TVA de 18 % s'ajouterait au montant que le client a réellement payé, et la comptabilité ne correspondrait plus à la caisse (problème constaté et corrigé lors des tests sur Odoo 18). Pour déclarer la TVA, créer dans Odoo une taxe « incluse dans le prix » et mettre son ID dans ODOO_LINE_TAX_IDS. Si les totaux diffèrent malgré tout, la commande n'est pas créée dans Odoo et l'erreur s'affiche dans l'administration.

Si Odoo est indisponible, **les commandes continuent de fonctionner** : elles sont marquées « échec » et se renvoient depuis Admin › Odoo › « Synchroniser maintenant », sans risque de doublon.

10. Sauvegardes et restauration

Deux choses sont à sauvegarder, et perdre l'une sans l'autre laisse un site incohérent (des plats dont la photo a disparu, des candidatures sans CV) : la base, et le volume des fichiers envoyés. Au minimum chaque jour, et avant chaque mise à jour.

Le dépôt fournit `scripts/backup.sh`, qui fait les deux : il exporte la base, archive le volume des fichiers, et supprime les sauvegardes de plus de 14 jours. Tout est écrit dans `~/sauvegardes`. Le lancer une fois à la main pour vérifier :

```
~/bonici/scripts/backup.sh
```

Puis l'automatiser chaque nuit à 3 h (`crontab -e`, une seule ligne) :

```
0 3 * * * $HOME/bonici/scripts/backup.sh >> $HOME/sauvegardes/backup.log 2>&1
```

Pourquoi un script plutôt qu'une longue ligne de cron
Une ligne de crontab ne peut pas être coupée en plusieurs lignes, et une commande de sauvegarde complète ne tient pas sur une ligne lisible : recopiée avec une coupure, elle ne s'exécute jamais, et rien ne le signale. Le script s'arrête au contraire à la première erreur et laisse une trace dans `backup.log`. Copier régulièrement `~/sauvegardes` hors du serveur : une sauvegarde sur la même machine ne protège pas d'une panne du serveur.

Restaurer

```
gunzip -c ~/sauvegardes/bonici-AAAA-MM-JJ.sql.gz \
  | docker compose -f compose.yaml exec -T db psql -U bonici -d bonici
```

Une sauvegarde jamais restaurée n'est pas une sauvegarde
La restaurer une fois sur une installation d'essai, avant d'en avoir besoin. C'est le seul moyen de savoir qu'elle est complète.

11. Exploitation courante

Besoin	Commande
Voir l'état des services	`docker compose -f compose.yaml ps`
Logs du site	`docker compose -f compose.yaml logs -f --tail=100 app`
Redémarrer le site	`docker compose -f compose.yaml restart app`
Renvoyer les commandes vers Odoo	`docker compose -f compose.yaml run --rm migrate npm run odoo:sync`
Espace disque Docker / nettoyage	`docker system df` · `docker image prune`

12. Points d'attention et dépannage

- **Le DNS avant le démarrage.** Si le domaine ne mène pas encore au serveur, Caddy ne peut pas obtenir le certificat et réessaie en boucle ; une fois le DNS propagé : `docker compose -f compose.yaml restart proxy`.
- **Le fichier .env n'est jamais copié dans l'image.** Après l'avoir modifié, relancer `docker compose -f compose.yaml --profile prod up -d` pour que les conteneurs le relisent.
- **Changer POSTGRES_PASSWORD après le premier démarrage ne suffit pas** : PostgreSQL l'a déjà enregistré dans le volume. Le changer aussi dans la base (`ALTER USER bonici PASSWORD '...'`) ou repartir d'un volume vide.
- **Sans SMTP**, les liens « mot de passe oublié » ne partent pas : ils s'affichent dans `logs app`. Le reste du site n'est pas affecté.
- **Les CV des candidatures sont des données personnelles** : ils ne sont accessibles qu'aux administrateurs, jamais par une adresse publique, et font partie de la sauvegarde des fichiers.

Symptôme	Cause probable et solution
Pas de HTTPS, erreur de certificat	DNS pas encore propagé, ou ports 80/443 fermés : vérifier `dig` et `sudo ufw status`, puis redémarrer proxy.
Page « Un problème est survenu »	La base est indisponible : `ps` puis `logs db` et `logs app`.
Le site ne démarre pas, migrate en erreur	`logs migrate` — le plus souvent un .env incomplet ou un mot de passe de base modifié (voir ci-dessus).
Commandes « échec » dans Admin › Odoo	Le message s'affiche sur la commande (clé API, ID produit, totaux) ; corriger puis « Synchroniser maintenant ».

Symptôme	Cause probable et solution
Un plat n'apparaît pas dans un restaurant	Admin › Restaurants › Carte : le plat doit être coché « à la carte » et « disponible ».

Annexe — Toutes les variables du fichier .env

Le corps du guide dit quoi faire, dans l'ordre. Cette annexe dit *tout* ce qui se règle, pour la personne qui reprend une installation existante. Une variable absente de ce tableau est une variable que l'application ne lit pas.

A.1 — Base de données

Variable	Ce qu'elle fait	Ce qu'il faut y mettre ici
POSTGRES_USER / **POSTGRES_DB**	Utilisateur et nom de la base, créés au premier démarrage.	bonici / bonici.
POSTGRES_PASSWORD	Mot de passe de la base.	**Obligatoire.** Aléatoire et long (5.3). Voir les points d'attention avant de le changer.
DATABASE_URL	Adresse de la base pour l'application.	Ignorée en production (construite par Docker). Sert au développement sans Docker.

A.2 — Site

Variable	Ce qu'elle fait	Ce qu'il faut y mettre ici
SITE_URL	Adresse publique, utilisée dans les liens des emails.	https://bonici.africa
DOMAIN	Domaine du certificat HTTPS (Caddy).	bonici.africa — www est redirigé automatiquement.
APP_PORT	Port local de l'application, derrière Caddy.	3000 par défaut. À changer seulement si le port est pris.
UPLOAD_DIR	Dossier des fichiers envoyés.	Fixé par Docker (/app/uploads, volume uploads). Ne pas modifier.
CONTACT_EMAIL	Email affiché sur la page Contact et dans le pied de page.	Facultatif. Vide : aucun email affiché.

A.3 — Premier administrateur

Variable	Ce qu'elle fait	Ce qu'il faut y mettre ici
ADMIN_EMAIL	Adresse du compte créé au premier démarrage.	Celle du responsable du site.
ADMIN_PASSWORD	Son mot de passe provisoire.	8 caractères minimum. Ignoré dès qu'un administrateur existe : le changer ensuite dans l'administration, pas ici.

A.4 — Email (mot de passe oublié)

Variable	Ce qu'elle fait	Ce qu'il faut y mettre ici
SMTP_HOST / **SMTP_PORT**	Serveur d'envoi.	Celui de l'hébergeur de messagerie ; port 587 (ou 465). Vide : les emails sont écrits dans les logs.
SMTP_USER / **SMTP_PASSWORD**	Compte d'envoi.	Chez Gmail : un mot de passe d'application, jamais celui du compte.
MAIL_FROM	Expéditeur affiché.	Bonici <no-reply@bonici.africa>

A.5 — Odoo (facultatif)

Variable	Ce qu'elle fait	Ce qu'il faut y mettre ici
ODOO_URL, **ODOO_DB**	Adresse et base Odoo.	Vides : intégration désactivée, les commandes restent sur le site.
ODOO_USERNAME, **ODOO_API_KEY**	Compte et clé API (ou mot de passe).	Un utilisateur ayant les droits Ventes.
ODOO_FALLBACK_PRODUCT_ID	Produit utilisé pour un plat sans ID produit Odoo.	ID du produit « Plat (site web) ».
ODOO_DELIVERY_PRODUCT_ID	Produit des frais de livraison.	ID du produit « Frais de livraison ».
ODOO_CONFIRM_ORDERS	Confirmer le bon dans Odoo ou le laisser en devis.	true (défaut).
ODOO_LINE_TAX_IDS	Taxes appliquées aux lignes.	Vide (aucune taxe ajoutée) ou l'ID d'une taxe incluse dans le prix (section 9).

Dépôt : https://github.com/D4Ron/Bonici · Guide généré par `docs/build-deployment-pdf.py` — le modifier et le relancer quand la procédure change, plutôt que de corriger le PDF à la main.